

Java Backend Architecture

Interview Series

Chapter 18.5

Rate Limiting & Security

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 18.5 – Rate Limiting & Security

Overview

Nginx provides multiple layers of abuse protection: request rate limiting (`limit_req`), connection limiting (`limit_conn`), IP access control, and WAF integration. These protect Java backends from DDoS, brute force, and resource exhaustion without application changes. Senior engineers must understand token bucket semantics and the burst/nodelay trade-off.

Key Definitions

Term	Definition
<code>limit_req_zone</code>	Shared memory zone tracking request rate per key. Uses leaky bucket algorithm.
<code>limit_req</code>	Applies rate limiting in a location. <code>burst=</code> allows queue; <code>nodelay</code> processes immediately.
<code>limit_conn_zone</code>	Shared memory tracking simultaneous connections per key.
<code>limit_conn</code>	Limits simultaneous connections per key in a location.
geo module	Creates a variable from client IP CIDR matching using radix tree (O(1) lookup).
allow / deny	IP access control. Evaluated top-to-bottom, first match wins. deny all as default.
auth_request	Makes subrequest to auth endpoint. 2xx allows, 401/403 denies the main request.
return 444	Nginx-specific: closes connection without HTTP response. Effective against bots.
ModSecurity	WAF module. OWASP Core Rule Set (CRS) blocks SQLi, XSS, CSRF patterns.
<code>\$binary_remote_addr</code>	Binary IP: 4 bytes IPv4, 16 bytes IPv6. 2x more memory-efficient than string in zones.

Diagram 1: Rate Limiting Leaky Bucket with Burst

```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s

Client sends 25 requests in 0.5 seconds:
|<-- burst=20 slots ----->| reject remaining 5 (429)
t=0.0s: req 1-5    -> process immediately (5 slots used)
t=0.1s: req 6-20   -> process immediately with nodelay (bucket full)
t=0.2s: req 21-25  -> REJECTED 429 Too Many Requests

Without nodelay: req 6-20 delayed to maintain 10r/s rate
With nodelay:    req 6-20 processed immediately, bucket depletes
```

Diagram 2: Multi-Layer Security at Nginx

```
Internet
|
v
[1] IP Allow/Deny (geo module)    -- block blacklisted IPs
[2] limit_conn (connection limiting) -- max simultaneous connections
[3] limit_req (rate limiting)     -- requests per second per IP
[4] auth_request (JWT validation) -- subrequest to auth service
[5] ModSecurity WAF              -- SQLi/XSS/CSRF blocking
|
v
Java Microservices (protected)
```

Code Examples

Rate Limiting Full Config:

```
http {
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
    limit_req_zone $binary_remote_addr zone=login:10m rate=1r/s;
    limit_req_status 429;
    limit_req_log_level warn;

    server {
        location /api/ {
            limit_req zone=api burst=20 nodelay;
            proxy_pass http://java_api;
        }
        location /auth/login {
            limit_req zone=login burst=5;
            proxy_pass http://java_auth;
        }
    }
}
```

Connection Limiting + IP Whitelisting:

```
geo $whitelist {
    default 0;
    10.0.0.0/8 1;
    192.168.0.0/16 1;
}

limit_conn_zone $binary_remote_addr zone=perip:10m;

location /api/ {
    limit_conn perip 10;
    limit_req zone=api burst=20 nodelay;
    proxy_pass http://java_api;
}
```

auth_request -- JWT Validation:

```
location /api/ {
    auth_request /auth-validate;
    auth_request_set $user_id $upstream_http_x_user_id;
    proxy_set_header X-User-Id $user_id;
    proxy_pass http://java_api;
}

location = /auth-validate {
    internal;
    proxy_pass http://auth_service/validate;
    proxy_pass_request_body off;
    proxy_set_header Content-Length '';
    proxy_set_header Authorization $http_authorization;
}
```

Interview Q&A – Rate Limiting & Security

Q1. Explain burst with and without nodelay.

Without nodelay: excess requests queue at the configured rate until burst exhausted, then rejected with 429. burst=20 rate=10r/s: 20 requests queue, each delayed 100ms. With nodelay: queued requests processed immediately consuming burst slots -- only requests exceeding burst rejected. nodelay is better for APIs where clients expect low-latency; without nodelay suits form submissions where delay is acceptable.

Q2. Why use \$binary_remote_addr instead of \$remote_addr for zones?

\$remote_addr is a string: 7-15 chars for IPv4, up to 39 for IPv6. \$binary_remote_addr is binary: 4 bytes IPv4, 16 bytes IPv6. In a 10MB zone, binary stores ~160k IPv4 entries vs ~120k for string. Also normalises IPv6 addresses ensuring consistent key lookup regardless of client address format differences.

Q3. How do limit_req and limit_conn complement each other?

limit_conn limits simultaneous connections per IP -- prevents connection exhaustion. limit_req limits request rate per IP -- prevents request flood. A client with 1 connection can still flood requests on that connection (HTTP/1.1 pipelining, HTTP/2 streams). Implement both: limit_conn perip 10 + limit_req zone=api rate=10r/s provides comprehensive protection.

Q4. What HTTP status should rate limiting return?

429 Too Many Requests (RFC 6585) is semantically correct. Set limit_req_status 429. Include Retry-After header for well-behaved clients: add_header Retry-After 60. Java API clients should implement exponential backoff on 429. Default limit_req_status is 503 -- change to 429 to distinguish rate limiting from server errors.

Q5. How does auth_request enable zero-trust security at Nginx?

auth_request makes a subrequest to an internal auth endpoint for every request. The Java auth service validates JWT/API key and returns 2xx (allow) or 401/403 (deny). Nginx passes request headers (Authorization), no body (proxy_pass_request_body off). Auth service returns headers (X-User-Id, X-Roles) that Nginx forwards downstream via auth_request_set + proxy_set_header. Centralises auth without SDK in every service.

Q6. Explain return 444 vs 403 for blocking bad actors.

return 403 sends an HTTP Forbidden response -- confirms to attackers the path exists and consumes Nginx resources to build the response. return 444 closes the TCP connection immediately with no response -- gives no information to attackers and uses minimal resources. Use 444 for known malicious IPs, empty User-Agents, and scanner fingerprints. Use 403 for legitimate access control failures.

Q7. How does the geo module differ from allow/deny for IP blocking?

allow/deny processed linearly O(n) per request. geo uses a radix tree: O(1) CIDR lookup, efficient for thousands of IP ranges. geo produces a variable enabling conditional logic (rate limit bypass, routing, logging). GeoIP2 module extends geo with country/city data for geographic access control without linear scan overhead.

Q8. How do you protect against Slowloris attacks at Nginx?

Slowloris keeps connections open by sending headers very slowly. Nginx mitigations: client_header_timeout 10s (disconnect if headers not received); client_body_timeout 10s; limit_conn perip 10 (cap connections per IP). Nginx's event-driven model is less vulnerable than threaded servers -- holds slow connections in epoll at near-zero cost vs Apache exhausting thread pool.

Q9. What is ModSecurity OWASP CRS and how does it work?

ModSecurity is a WAF module for Nginx. OWASP Core Rule Set contains rules matching: SQL injection (UNION SELECT patterns), XSS (script tag patterns), path traversal (../etc/passwd), command injection. Paranoia levels 1-4 control strictness (higher = more false positives). Start with DetectionOnly mode (logs without blocking), tune false positives against Java Spring app, then enable blocking mode (SecRuleEngine On).

Q10. How do you implement API key validation at Nginx?

Simple: map \$http_x_api_key \$valid { default 0; 'key1' 1; 'key2' 1; } if (\$valid = 0) { return 403; } -- keys in config (needs reload to add keys). Better: auth_request to a Java validation service checking keys in database. Best: JWT with RS256 -- validate signature using public key at Nginx (lua-resty-jwt in OpenResty) without subrequest latency. Stateless, no DB lookup required.

Q11. Explain the satisfy directive for flexible access control.

satisfy all (default): client must pass ALL checks (auth_basic AND allow/deny). satisfy any: passing ANY check is sufficient. Pattern: internal IPs bypass JWT auth, external IPs must authenticate. geo \$is_internal + auth_request. satisfy any: internal traffic

skips auth overhead, external traffic enforced. Avoids auth latency for internal Java service calls.

Q12. How do you rate limit per authenticated user rather than per IP?

After auth_request validates JWT, auth service returns X-User-Id header. auth_request_set \$userid \$upstream_http_x_user_id; then create a zone keyed by user: limit_req_zone \$userid zone=user_api:10m rate=100r/s. This rate limits per user account regardless of IP (handles NAT/proxies) and allows different rate limits per user tier by using different zones based on user role.